

# Week 6 In-Class Exercises (More on Lists)

## Q1: List of Numbers

### Part (a) [ \* ]

Define a function called `get_larger_values()`. This function takes in a list of float values as its parameter. The function **returns** a new list of float values, which are those values from the original list which are above the average of the values in the original list. The original list should remain unchanged.

You can assume that the original list contains at least one value.

For example, `get_larger_values([2.5, 3.5, 5.5, 1.0])` should return the list `[3.5, 5.5]`, because the average of the values in the original list is 3.125 ( $(2.5 + 3.5 + 5.5 + 1.0) / 4 = 3.125$ ), and 3.5 and 5.5 are the two values in the original list that are above 3.125.

### Part (b) [ \*\* ]

Define a function called `merge_list()`. The function takes in two lists of numbers as its two parameters. It merges the two lists by alternating between the two lists to take their values one by one and inserting these values into a new list. It then returns the new list. The two original lists should remain unchanged.

The two lists may not be of the same length. The extra elements from the longer list are added to the returned list without merging with any element from the shorter list.

For example, `merge_list([1, 3, 10, 15, 4, 7, 12], [9, 5, 2])` returns the list `[1, 9, 3, 5, 10, 2, 15, 4, 7, 12]`. Note that 15, 4, 7 and 12 (the extra elements from the longer list) are included in the returned list at the end.

### Part (c) [ \*\* ]

Define a function called `check_numbers()`. The function takes in two lists of positive integers (called `int_list_1` and `int_list_2`) as its two parameters. You can assume that both lists are not empty, and contain only positive integers. The function returns `True` if each integer in `int_list_1` is divisible by at least one of the integers in `int_list_2`; otherwise it returns `False`.

For example, `check_numbers([3, 8, 10, 15, 16], [9, 3, 2, 5])` returns `True`, while `check_numbers([3, 8, 10, 6, 2, 5], [9, 3, 7])` returns `False`.

## Q2: Books [ \*\* ]

### Part (a)

You are given a list of tuples, where each tuple represents a copy of a book in a library's collection. Specifically, each tuple consists of four elements:

- (1) A string representing the title of the book.
- (2) A string indicating the edition of the book. E.g., the third edition of a book would have this string set to "Ed-3".
- (3) A string indicating whether this book is a hardcover book or a paperback book, i.e., the value of this string is either "hardcover" or "paperback".
- (4) An integer indication the number of copies of the book with the specified title, edition and type (hardcover vs. paperback).

Note that the library has multiple books having the same title.

Write a function called `get_unique_titles()` that takes in such a list of tuples as its parameter. The function returns a new list of strings, which are the unique book titles from the original list.

For example, `get_unique_titles([("Intro to Programming", "Ed-2", "paperback", 2), ("Intro to Python", "Ed-1", "paperback", 5), ("Intro to Programming", "Ed-3", "hardcover", 4)])` should return the list `["Intro to Programming", "Intro to Python"]`.

### Part (b)

Write a function called `get_titles_and_counts()`. The function takes in the same kind of list of books as described above. The function should return a list of tuples, where each tuple contains two elements: (1) A book title, and (2) the number of copies of that book.

For example, `get_titles_and_counts([("Intro to Programming", "Ed-2", "paperback", 2), ("Intro to Python", "Ed-1", "paperback", 5), ("Intro to Programming", "Ed-3", "hardcover", 4), ("Intro to Python", "Ed-3", "hardcover", 3)])` should return the list `[("Intro to Programming", 6), ("Intro to Python", 8)]`.

We get this result because there are six copies of "Intro to Programming":

- Two in a 2<sup>nd</sup> edition in paperback and
- Four in a 3<sup>rd</sup> edition in hardcover

and there are eight copies of "Intro to Python":

- Five in a 1<sup>st</sup> edition in paperback and
- Three in a 3<sup>rd</sup> edition in hardcover.

### Q3: Retrieve Numbers [ \*\* ]

Define a function called `retrieve_numbers()`. The function takes in a string that contains some numbers as its parameter. The function **returns** the sequence of these numbers separated by spaces as a string.

For example,

- `retrieve_numbers("12abc600$##0900AB 100X")`  
returns the string "12 600 0900 100"
- `retrieve_numbers("34.5689abc980")`  
returns the string "34 5689 980"
- `retrieve_numbers("xyz")`  
returns the (empty) string ""
- `retrieve_numbers("abc25xyz")`  
returns the string "25"